

"Idea Se Production-Ready Blueprint"

Document:	Blueprint Generator — Feature PRD v1.0
Feature:	Deep Dive — Complete Project Blueprint
Date:	June 2026
Status:	Final — Ready for Development

1. Feature Overview

Blueprint Generator **lazy-loaded deep-dive** hai. User jab ek idea select karta hai, tabhi trigger hota hai. Output: **7-section comprehensive project blueprint** — PRD, Tech Stack, Database Schema, API Design, User Flow, Implementation Plan, aur Pitch Deck Content. Har section user ki profile ke hisaab se personalized hota hai.

Key Principle: Same idea, different user = different blueprint. Beginner ko simple MVP plan, Advanced ko microservices architecture. Budget \$0 = free tiers only, \$5000 = scalable cloud.

2. Trigger Condition & User Flow

Trigger: **ONLY** when user clicks 'Select' on an idea card

- **Step 1:** User idea card pe click karta hai → 'Select Idea' button press karta hai
- **Step 2:** System selected idea ka ID store karta hai, status 'SELECTED' mark karta hai
- **Step 3:** BlueprintArchitectAgent trigger hota hai (background/async)
- **Step 4:** Loading state dikhta hai: 'Generating Blueprint...' with DNA helix animation
- **Step 5:** Blueprint complete hone pe UI mein 7 tabs appear: Overview, PRD, Tech Stack, Database, API, User Flow, Timeline
- **Step 6:** User koi bhi tab open kar sakta hai, content accordion-style expand hota hai
- **Step 7:** Export buttons: Markdown, PDF, or Notion integration

3. Blueprint — 7 Sections Detail

Section 1: Project Overview

2-3 sentence elevator pitch. Problem + Solution + Target audience + Unique value proposition. User profile ke hisaab se tone adjust hota hai (beginner = simple language, advanced = technical depth).

Section 2: Product Requirements Document (PRD)

Sub-sections: Objectives (SMART goals), Features (with priority: MUST/SHOULD/COULD), User Stories (As a [user], I want [feature], so that [benefit]), Acceptance Criteria (given/when/then format).

Section 3: Tech Stack

5 categories: Frontend, Backend, Database, AI/ML, DevOps/Testing. Har technology ke saath 'Why this?' justification. Beginner: HTML/CSS/JS + GitHub Pages. Advanced: Next.js + tRPC + PostgreSQL + Docker + AWS.

Section 4: Database Schema

Tables, fields, types, relations, indexes. Prisma schema format mein output. ER diagram ke liye Mermaid.js code bhi generate karta hai. Foreign keys, constraints, aur indexing strategy included.

Section 5: API Design

REST endpoints with: Path, Method, Auth requirement, Request payload schema, Response schema, Error codes, Rate limiting. OpenAPI/Swagger format mein exportable. tRPC routers ke liye type-safe definitions.

Section 6: User Flow

Step-by-step user journey: Screen names, user actions, system responses, decision points. Mermaid.js flowchart code generate karta hai. 3 paths: Happy path, Error path, Edge cases.

Section 7: Implementation Plan

Phases with duration, milestones, dependencies, and deliverables. Gantt chart ke liye data structure. Risk assessment per phase. Team role assignment (if team size > 1).

4. BlueprintArchitect Agent

```
// BlueprintArchitect System Prompt You are a Senior Technical Architect with 15+
years experience. Generate a complete, production-ready project blueprint for the
given idea. Input: - Selected Idea: {selected_idea} - User Profile: {user_profile}
- Mode: {mode} // GENERATOR, EVOLUTION, or ARCHAEOLOGY Personalization Rules: -
Skill Level: {skill_level} → Adjust complexity - BEGINNER: Monolith, no framework,
client-side where possible, free hosting - INTERMEDIATE: Frameworks allowed, simple
backend, basic database - ADVANCED: Microservices, event-driven, CI/CD,
cloud-native - Budget: ${budget} → - $0: GitHub Pages, Supabase free tier, Vercel
hobby, no paid APIs - $100-500: Basic cloud, shared hosting, essential paid
services - $500+: Dedicated servers, managed services, premium APIs - Team Size:
{team_size} → - 1: Solo-friendly stack, minimal coordination - 2-3: Simple
collaboration tools, basic branching strategy - 5+: Full CI/CD, code review,
documentation, project management - Timeline: {timeline} → - 2 weeks: MVP only,
core features, no polish - 1 month: MVP + 2-3 enhancements - 3+ months: Full feature
set, testing, optimization - Tech Preferences: {tech_prefs} → Prioritize these
technologies Output: Strict JSON with all 7 sections. Each section must be detailed
enough for a developer to start coding immediately.
```

5. Complete Output Example

```
{ "overview": "AI Internship Matchmaker is a web platform that uses AI to match college students with relevant internships based on their skills, interests, and location. It reduces application time by 80% and increases match accuracy to 85%.", "prd": { "objectives": [ "Automate internship matching with 85% accuracy within 3 months", "Reduce average application time from 2 hours to 15 minutes" ], "features": [ { "name": "Skill Gap Analysis", "description": "AI analyzes student skills vs job requirements and shows missing skills", "priority": "MUST", "effort": "MEDIUM" }, { "name": "Auto-Apply", "description": "One-click application to matched internships", "priority": "SHOULD", "effort": "HIGH" } ], "userStories": [ "As a student, I want to upload my resume, so that the system can analyze my skills", "As a student, I want to see matched internships, so that I don't waste time on irrelevant applications" ], "acceptanceCriteria": [ "Given a resume with React skills, when analyzed, then system shows 5+ React internships", "Given a matched internship, when user clicks apply, then application is sent within 10 seconds" ] }, "techStack": { "frontend": { "name": "Next.js 14", "version": "14.2.0", "reason": "SSR for SEO, API routes for backend, App Router for modern patterns" }, "backend": { "name": "Next.js API Routes + tRPC", "reason": "Type-safe APIs, same codebase as frontend, reduces context switching" }, "database": { "name": "PostgreSQL + Prisma", "reason": "Relational data for matching algorithm, type-safe ORM" }, "ai": { "name": "OpenAI GPT-4 + text-embedding-3-small", "reason": "Resume parsing, skill extraction, semantic matching" }, "devops": { "name": "Vercel + GitHub Actions", "reason": "Free tier for hosting, automated CI/CD" } }, "databaseSchema": { "tables": [ { "name": "students", "fields": [ { "name": "id", "type": "UUID", "primary": true }, { "name": "email", "type": "String", "unique": true }, { "name": "skills", "type": "JSONB", "index": true, "description": "Extracted skills from resume" }, { "name": "location", "type": "String", "index": true }, { "name": "resumeText", "type": "Text" } ], "relations": [ { "table": "applications", "type": "ONE_TO_MANY", "field": "studentId" } ] }, { "name": "internships", "fields": [ { "name": "id", "type": "UUID", "primary": true }, { "name": "title", "type": "String" }, { "name": "requiredSkills", "type": "JSONB", "index": true }, { "name": "location", "type": "String", "index": true }, { "name": "companyId", "type": "UUID", "foreignKey": "companies.id" } ] } ], "indexes": [ { "name": "idx_skills_gin", "table": "students", "fields": ["skills"], "type": "GIN" }, { "name": "idx_location", "table": "internships", "fields": ["location"], "type": "BTree" } ] }, "apiDesign": { "baseUrl": "/api", "version": "v1", "endpoints": [ { "path": "/students", "method": "POST", "auth": "JWT", "description": "Register new student", "request": { "email": "string", "password": "string", "resumeText": "string" }, "response": { "id": "UUID", "email": "string", "createdAt": "datetime" }, "errors": ["400: Invalid email", "409: Email already exists"] }, { "path": "/match", "method": "POST", "auth": "JWT", "description": "Get matched internships for student", "request": { "studentId": "UUID", "limit": "number?" }, "response": { "matches": [ { "internshipId": "UUID", "matchScore": "number", "reason": "string" } ] }, "errors": ["401: Unauthorized", "404: Student not found"] } ], "rateLimiting": "100 requests/minute per user" }, "userFlow": [ { "step": 1, "screen": "Landing", "action": "User visits site", "system": "Show value proposition" }, { "step": 2, "screen": "Sign Up", "action": "User enters email + password", "system": "Validate + create account" }, { "step": 3, "screen": "Upload Resume", "action": "User pastes resume text", "system": "AI extracts skills" }, { "step": 4, "screen": "Dashboard", "action": "User views matched internships", "system": "Show top 10 matches with scores" }, { "step": 5, "screen": "Internship Detail", "action": "User clicks on internship", "system": "Show details + apply button" }, { "step": 6, "screen": "Apply", "action": "User clicks apply", "system": "Send application + confirmation" } ], "implementationPlan": { "phases": [ { "name": "Phase 1: Foundation", "duration": "1 week", "tasks": ["Setup Next.js project", "Configure Prisma + PostgreSQL", "Implement Auth (NextAuth)", "Create basic layout"], "deliverables": ["Working auth flow", "Database connected", "Basic UI"] }, { "name": "Phase 2: Core Features", "duration": "1 week", "tasks": ["Resume
```

```
upload + parsing", "Skill extraction (OpenAI)", "Basic matching algorithm",  
"Internship listing"], "deliverables": ["Resume parser working", "Match algorithm  
v1", "Internship CRUD"] }, { "name": "Phase 3: Polish", "duration": "3 days",  
"tasks": ["UI polish", "Error handling", "Testing", "Deploy to Vercel"],  
"deliverables": ["Production-ready app", "Deployed URL"] } ], "risks": [ {"risk":  
"OpenAI API rate limits", "mitigation": "Implement caching + fallback keyword  
matching"}, {"risk": "Resume parsing accuracy", "mitigation": "Manual skill input  
as fallback"} ] } }
```

6. Database Schema

```
model Blueprint { id String @id @default(uuid()) ideaId String @unique idea Idea
@relation(fields: [ideaId], references: [id]) projectId String @unique project
Project @relation(fields: [projectId], references: [id]) // 7 sections overview
String prd Json // {objectives, features[], userStories[], acceptanceCriteria[]}
techStack Json // {frontend, backend, database, ai, devops} databaseSchema Json //
{tables[], indexes[], relations[]} apiDesign Json // {baseUrl, version,
endpoints[], rateLimiting} userFlow Json // {step, screen, action, system}[]
implementationPlan Json // {phases[], risks[]} // Export formats markdown String //
Complete markdown document openApiSpec String? // Swagger/OpenAPI YAML (if
applicable) mermaidDiagrams Json? // {erDiagram, userFlow, architecture} createdAt
DateTime @default(now()) }
```


7. Export Formats

Markdown

Pure text format. Developer directly copy-paste karke GitHub repo mein README.md bana sakta hai.

PDF

Styled PDF with logo, colors, sections. Stakeholder presentation ke liye.

OpenAPI/Swagger

API section se auto-generated YAML. Postman ya Swagger UI mein import kar sakta hai.

Mermaid.js

Database ER diagram, user flow chart, aur architecture diagram ke liye code. GitHub natively support karta hai.

Notion Integration

Direct Notion page create karta hai with sections as blocks. Future feature.

8. Interview Talking Points

Why 7 Sections Instead of a Generic Document?

Generic PRD mein sab mix hota hai. 7 sections se developer directly apne need ka section padhta hai: Frontend dev → Tech Stack + API, Backend dev → Database + API, PM → PRD + User Flow, Founder → Overview + Implementation Plan. Modular = usable.

How Does Personalization Work?

Same 'AI Resume Matcher' idea, 3 different users: Beginner (\$0, solo, 2 weeks) → HTML/CSS/JS, client-side, no AI API (keyword matching), GitHub Pages. Intermediate (\$100, 2 people, 1 month) → React + Node + MongoDB, basic OpenAI integration, Vercel. Advanced (\$5000, 5 people, 3 months) → Next.js + tRPC + PostgreSQL + vector search + microservices + AWS. Blueprint har baar different.

Why Lazy-Loaded?

5 ideas × 7 sections × detailed output = ~50,000 tokens. GPT-4 pricing: \$0.03/1K input + \$0.06/1K output. Ek user ke liye \$15-20 lag sakta hai agar sab ideas ka blueprint generate karein. Lazy loading se sirf selected idea → \$3-4. 80% cost saving.

What Makes This 'Production-Ready'?

Blueprint mein sirf 'use React' nahi — 'Next.js 14.2.0, App Router, SSR for SEO, API routes for unified codebase' — specific versions, specific reasons. Database mein indexes specified hain. API mein rate limiting defined hai. Implementation plan mein risks + mitigations hain. Ye copy-paste karke coding start kar sakta hai.

— End of Guide 4: Blueprint Generator PRD —